



PROJETO MINI ABP

Sistema de Gestao de Leads

Modelagem, populacao e consultas com MongoDB

1000 Valle Multimarcas · Banco de Dados Nao Relacional



O que é o projeto e por que MongoDB

O PROJETO

- A 1000 Valle Multimarcas é uma revendedora de veículos com múltiplas unidades.
- O processo comercial começa com a geração de um lead - um potencial cliente interessado num veículo.
- O contato chega por vários canais: visita, telefone, WhatsApp, Instagram e formulários digitais.
- O sistema associa o lead ao cliente, vincula loja e atendente, registra status/estágio e gera indicadores gerenciais.



POR QUE MONGODB?



Orientado a documentos Banco NoSQL que guarda dados em documentos JSON/BSON.



Estrutura natural Representa dados hierárquicos sem a rigidez do modelo relacional.



Flexível e escalável Flexibilidade no schema, escalabilidade horizontal.



Alto desempenho Ótimo para grandes volumes de leitura e escrita.



leadsdb — as 6 colecoes do sistema



users

Atendentes, gerentes e admin do sistema.



clientes

Clientes vinculados aos leads (collection customers).



lojas

Unidades / lojas onde o lead e atendido (stores).



leads

Coracao do sistema: liga cliente, loja e atendente.



negociacoes

Evolucao da venda, com historico de estagios.



logs

Registro de acoes dos usuarios no sistema.



Referencing — referencia por ObjectId

O que é? Em vez de copiar os dados, o documento guarda apenas o **ObjectId** da entidade relacionada (como uma chave estrangeira).

Campo	Colecao referenciada
leads.customerId	customers
leads.storeId	stores
leads.attendantId	users
customers.createdBy	users
negotiations.leadId	leads
logs.userId	users

Tabela 1 — Mapeamento de referencias entre colecoes

POR QUE REFERENCING?

Consistencia dos dados

users, customers e stores mudam com frequencia. Duplicar geraria inconsistencias — mudar o nome de um atendente exigiria atualizar todos os documentos.

Ciclo de vida independente

Cada entidade pode ser criada, consultada e removida sem depender dos documentos que a referenciam.

Reutilizacao

Um mesmo usuario pode ser atendente de varios leads e autor de varios logs. Embutir geraria redundancia.



Documento embutido e buscas diretas



EMBEDDING — documento embutido

Aplicado na coleção negotiations, para guardar o histórico de estágios da negociação num array history dentro do próprio documento.

```
history: [  
  { stage: "contato_inicial", status: "aberta" },  
  { stage: "proposta_enviada", status: "em_andamento" },  
  { stage: "fechamento", status: "encerrada" }  
]
```

- ✓ **Pertencimento exclusivo** — o histórico só existe dentro daquela negociação.
- ✓ **Localidade de leitura** — lido junto com a negociação, sem join extra.
- ✓ **Crescimento limitado** — poucas transições; longe do limite de 16 MB.



Quando usar find()

Para buscas diretas e pontuais, com filtros simples, sem agrupar, calcular ou juntar coleções.

- Busca de leads por canal de origem (channel)
- Clientes criados por um determinado usuário
- Negociações abertas ou por status específico
- Logs de ações de um usuário

Por que find()? Eficiente em leituras diretas; com índices evita o full collection scan. Mais simples de escrever, ler e manter.



Filtrando com find() — o WHERE do MongoDB

SQL

```
SELECT * FROM clientes WHERE idade > 18
```



MongoDB

```
db.clientes.find({ idade: { $gt: 18 } })
```

\$gt / \$lt

Maior que / menor que

```
{ idade: { $gt: 18 } }
```

Leads com pontuacao acima de um limite.

\$and

Todas as condicoes

```
{ $and: [ {ativo:true}, {idade:{$gt:18}} ] }
```

Cliente ativo E maior de idade.

\$or

Qualquer condicao

```
{ $or: [ {canal:"site"}, {canal:"whatsapp"} ] }
```

Leads do site OU do WhatsApp.

\$exists

Campo existe?

```
{ telefone: { $exists: true } }
```

So clientes que tem telefone cadastrado.

Projecao

Escolher campos

```
find(...), { nome:1, idade:1, _id:0 }
```

Traz so nome e idade no resultado.

sort / skip / limit

Ordenar e paginar

```
.sort({idade:-1}).skip(10).limit(5)
```

Ordena e pagina os resultados.



O que é o Aggregation Pipeline

Processa documentos numa sequencia de etapas (stages): a saida de uma etapa é a entrada da proxima. Ideal para agrupar, calcular metricas, juntar colecoes e projetar campos — tudo o que o `find()` não faz.



5 AGREGACOES, 5 PERGUNTAS DE NEGOCIO

- 1 Leads por canal de origem
- 2 Negociacoes por status
- 3 Taxa de conversao
- 4 Leads por atendente
- 5 Negociacoes por importancia



Leads por canal de origem

Conta quantos leads vieram de cada canal (WhatsApp, site, indicacao...) e ordena do maior volume para o menor.

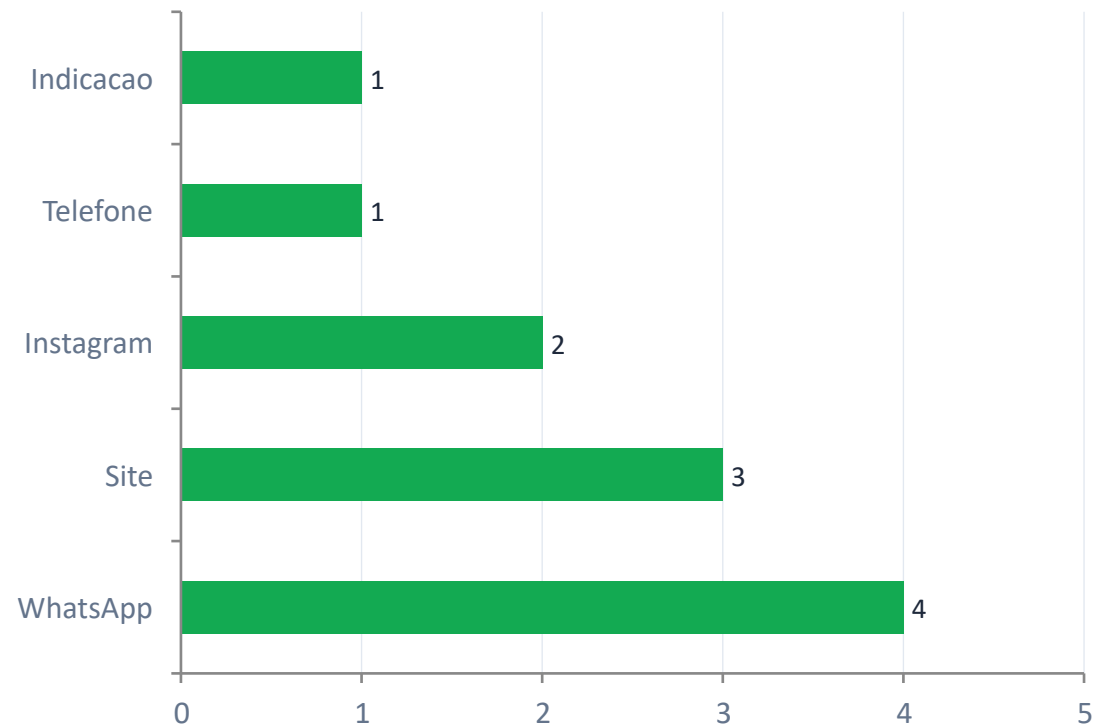
```
db.leads.aggregate([
  { $group: { _id: "$channel",
               total: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $project: { origem: "$_id",
                 total: 1, _id: 0 } }
])
```

\$group + \$sum Unica forma nativa de contar documentos por categoria. O find() nao agrupa nem conta.

\$sort: { total: -1 } Ordena de forma descendente, facilitando a leitura do canal mais relevante.

\$project Renomeia _id para origem, deixando o resultado mais legivel.

Resultado tipico — leads por canal



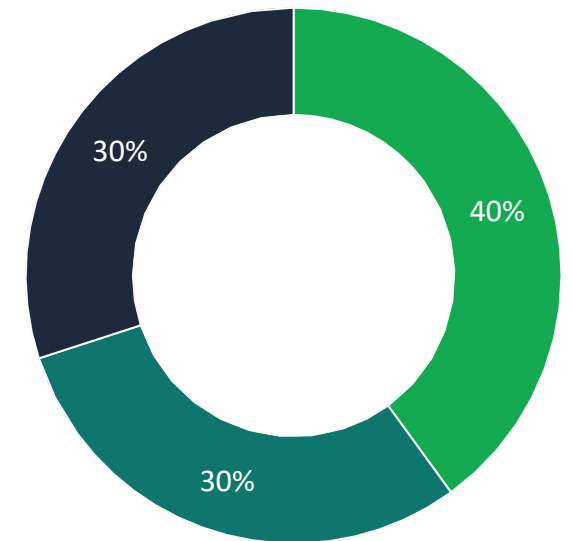


Negociacoes por status

Distribui as negociacoes por status (aberta, em andamento, encerrada), dando uma visao geral do funil de vendas.

```
db.negotiations.aggregate([
  { $group: { _id: "$status",
              total: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $project: { status: "$_id", total: 1, _id: 0 } }
])
```

Distribuicao das negociacoes no funil



■ Aberta ■ Em andamento ■ Encerrada



Por que consultar negotiations e nao leads?

O status da negociacao reside na colecao **negotiations**, e nao em **leads**. Consultar a colecao correta evita lookups (joins) desnecessarios.

\$group + \$sum continuam obrigatorios para consolidar a contagem por categoria.



Taxa de conversao

Calcula a porcentagem de negociacoes encerradas (convertidas) sobre o total — a principal metrica de desempenho do sistema de leads.

```
db.negotiations.aggregate([
  { $group: { _id: null,
    total: { $sum: 1 },
    encerradas: { $sum: { $cond: [
      { $eq: ["$status", "encerrada"] }, 1, 0 ] } } },
  { $project: { total: 1, encerradas: 1,
    taxaConversao: { $multiply: [
      { $divide: ["$encerradas", "$total"] }, 100 ] } } }
])
```

TAXA DE CONVERSAO

30%

3 encerradas de 10 negociacoes (exemplo)

\$cond dentro do \$group

Conta condicionalmente (so 'encerrada') sem precisar de um segundo \$match.

Uma unica passagem

total e encerradas sao acumulados juntos — menos passagens pelo pipeline, mais eficiencia.

\$divide e \$multiply

No \$project: divide encerradas/total e multiplica por 100 para obter a porcentagem.



Leads por atendente — o papel do \$lookup

Conta quantos leads cada atendente possui e mostra o NOME do atendente no resultado, em vez de apenas o ObjectId.

```
db.leads.aggregate([
  { $group: { _id: "$attendantId",
              total: { $sum: 1 } } },
  { $lookup: { from: "users",
              localField: "_id",
              foreignField: "_id", as: "user" } },
  { $unwind: "$user" },
  { $sort: { total: -1 } },
  { $project: { atendente: "$user.name",
                total: 1, _id: 0 } }
])
```



Por que o join? — Como o modelo usa referencin, leads guarda apenas o attendantId. Para mostrar o nome, e preciso juntar com a colecao users.



\$lookup — Faz a juncao dentro do proprio pipeline, sem uma segunda consulta na aplicacao.



\$unwind — Desaninha o array gerado pelo lookup, permitindo que o \$project acesse user.name diretamente.

Abordagem recomendada pelo MongoDB para joins pontuais entre colecoes.



Negociacoes por importancia

Distribui as negociacoes pelo campo importance (alta, media, baixa), permitindo ver a proporcao de leads prioritarios.

```
db.negotiations.aggregate([
  { $group: { _id: "$importance",
              total: { $sum: 1 } } },
  { $sort: { total: -1 } },
  { $project: { importancia: "$_id", total: 1, _id: 0 } }
])
```

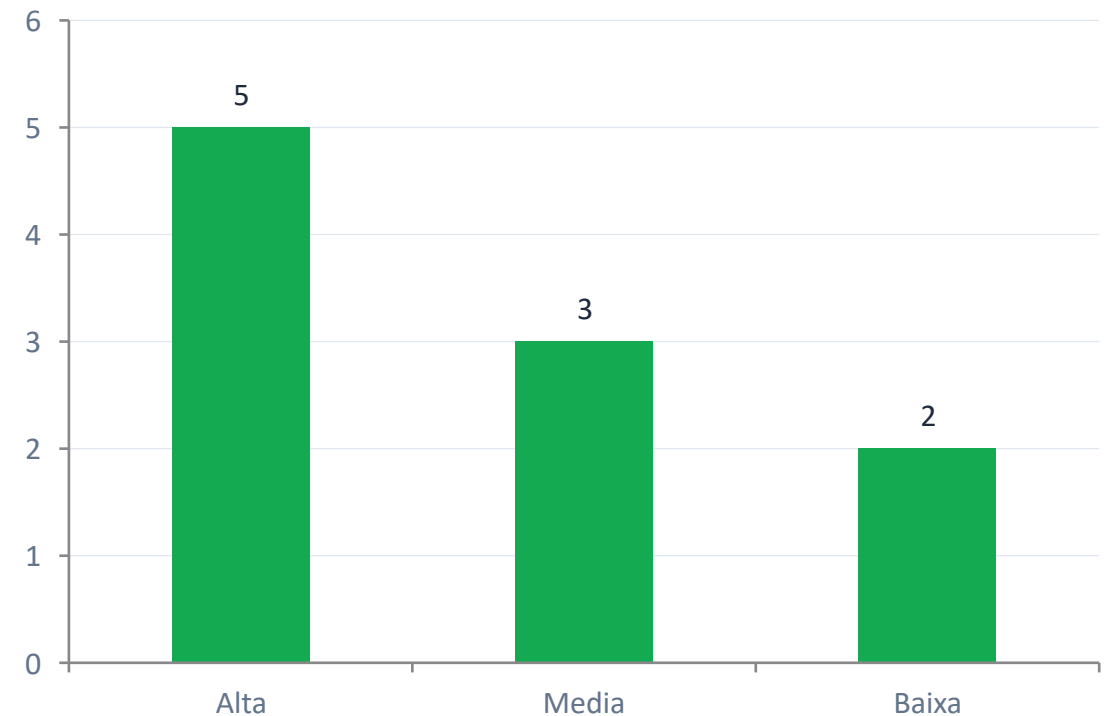


Por que ate consultas simples precisam do pipeline?

Mesmo sendo estruturalmente simples, esta consulta exige **\$group** e **\$sort** — operacoes que o `find()` nao suporta.

O Aggregation Pipeline e a unica forma nativa de responder, sem processar dados na aplicacao.

Negociacoes por importancia





Consideracoes finais



Modelagem guiada pelo acesso

Documentos lidos sempre juntos foram embutidos; entidades com ciclo de vida proprio foram referenciadas.



find() vs. Pipeline

find() para buscas pontuais e diretas; Aggregation Pipeline para agrupar, calcular e juntar dados.



Resultado

Consultas eficientes, dados consistentes e colecoes que refletem o dominio de gestao de leads.